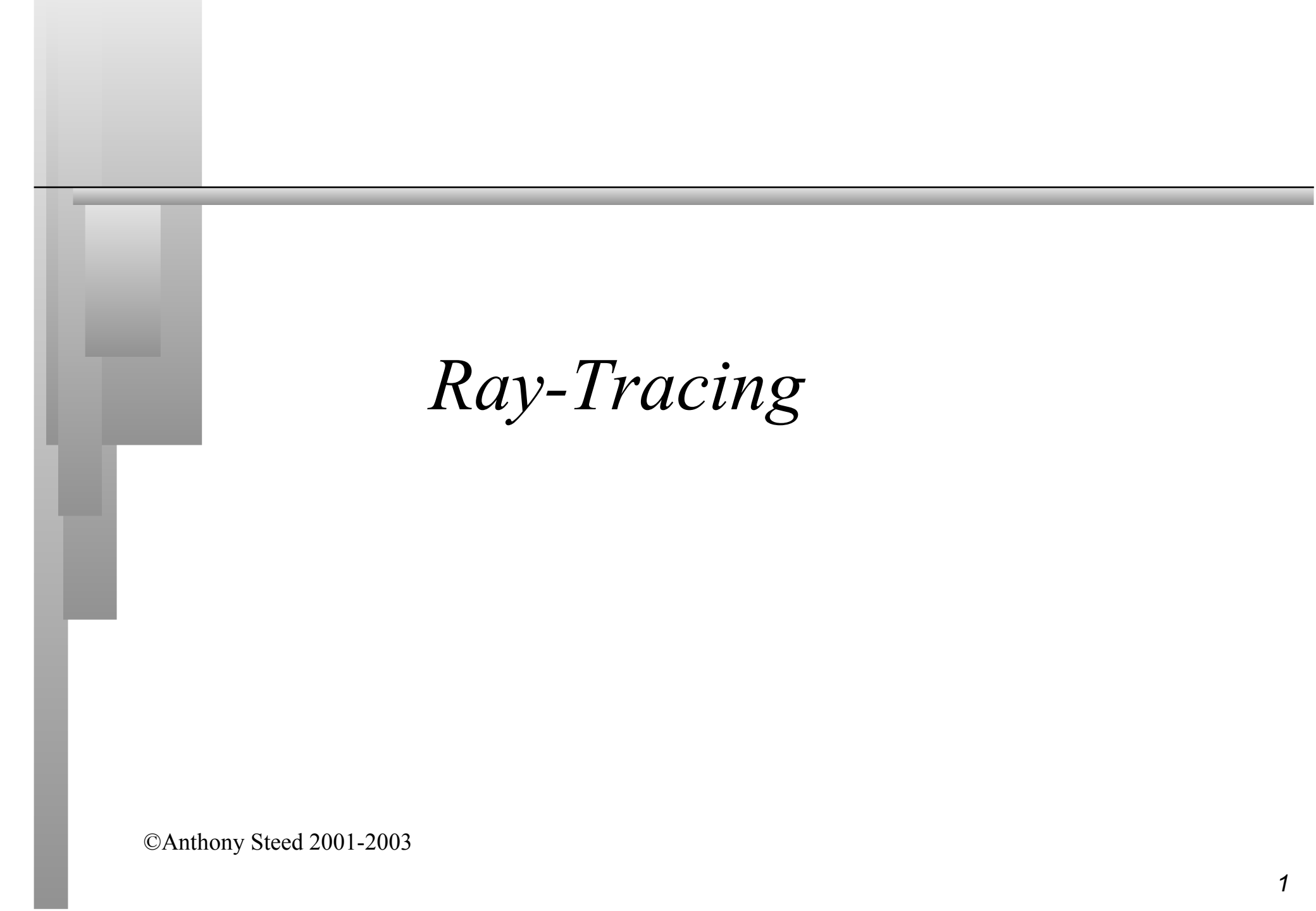


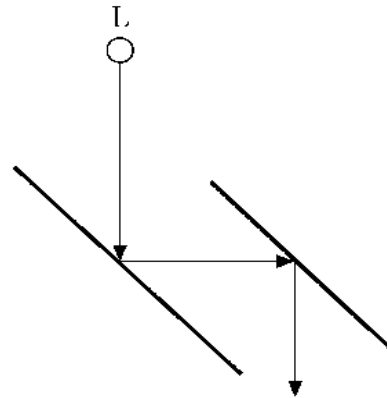
A decorative graphic consisting of a thick gray vertical bar on the left side of the slide and a thin gray horizontal line extending from the vertical bar across the top of the slide.

Ray-Tracing

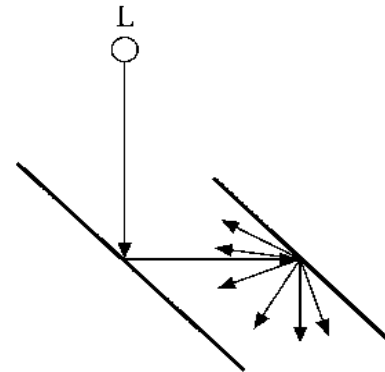
Overview

- *Recursive Ray Tracing*
- *Shadow Feelers*
- *Snell's Law for Refraction*
- *When to stop!*

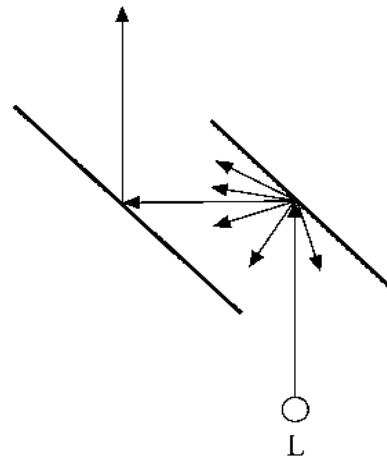
Recap: Light Transport



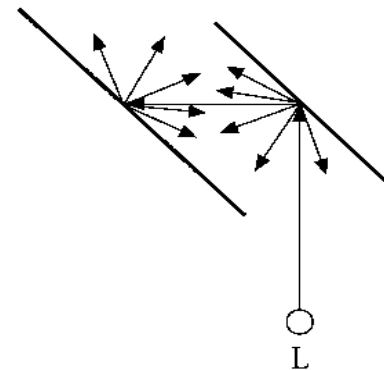
(a) specular to specular



(b) specular to diffuse



(c) diffuse to specular



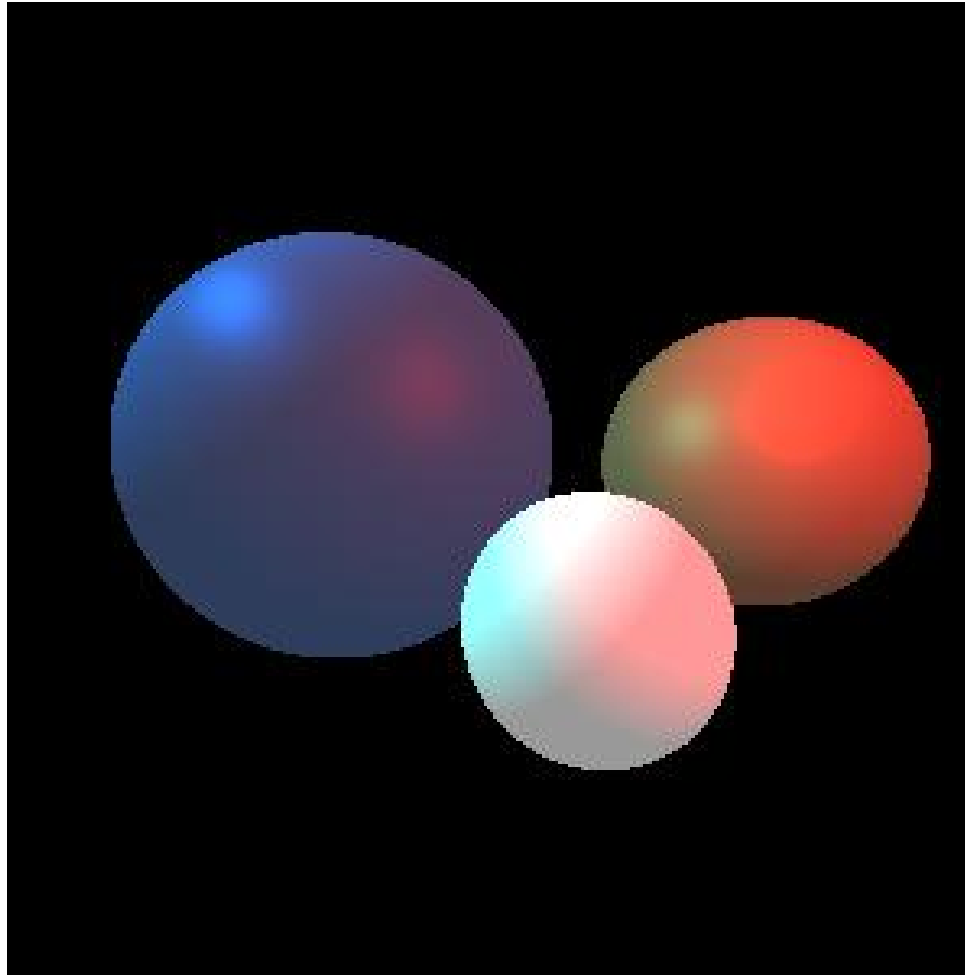
(d) diffuse to diffuse

Recap: Local Illumination

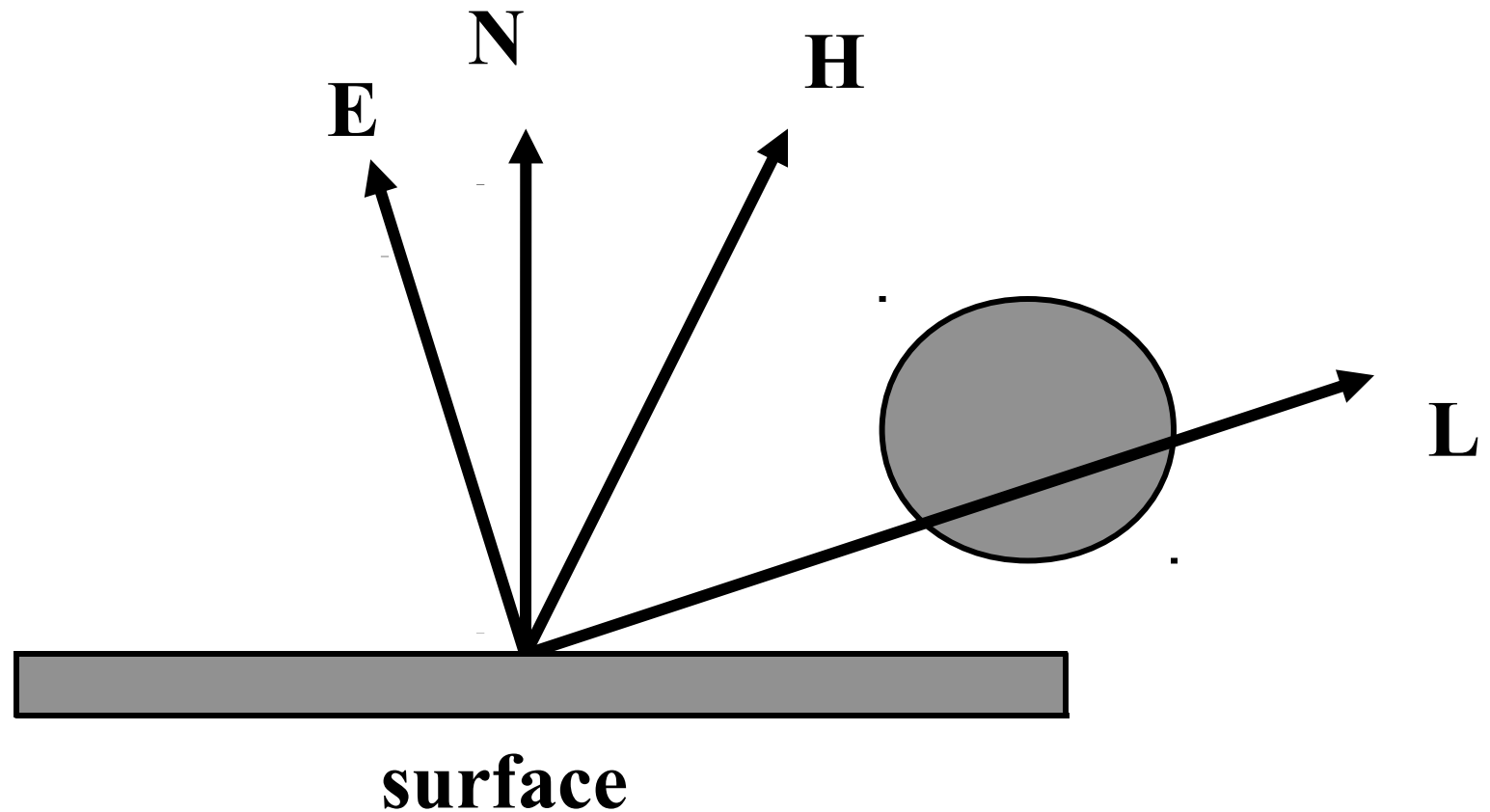
$$I_r = k_a I_a + \sum I_{i,j} \left(k_d (n \cdot l_j) + k_s (h_j \cdot n)^m \right)$$

- *Ambient, diffuse & specular components*
- *The sum is over the specular and diffuse components for each light*

Recap: Result of Ray Casting



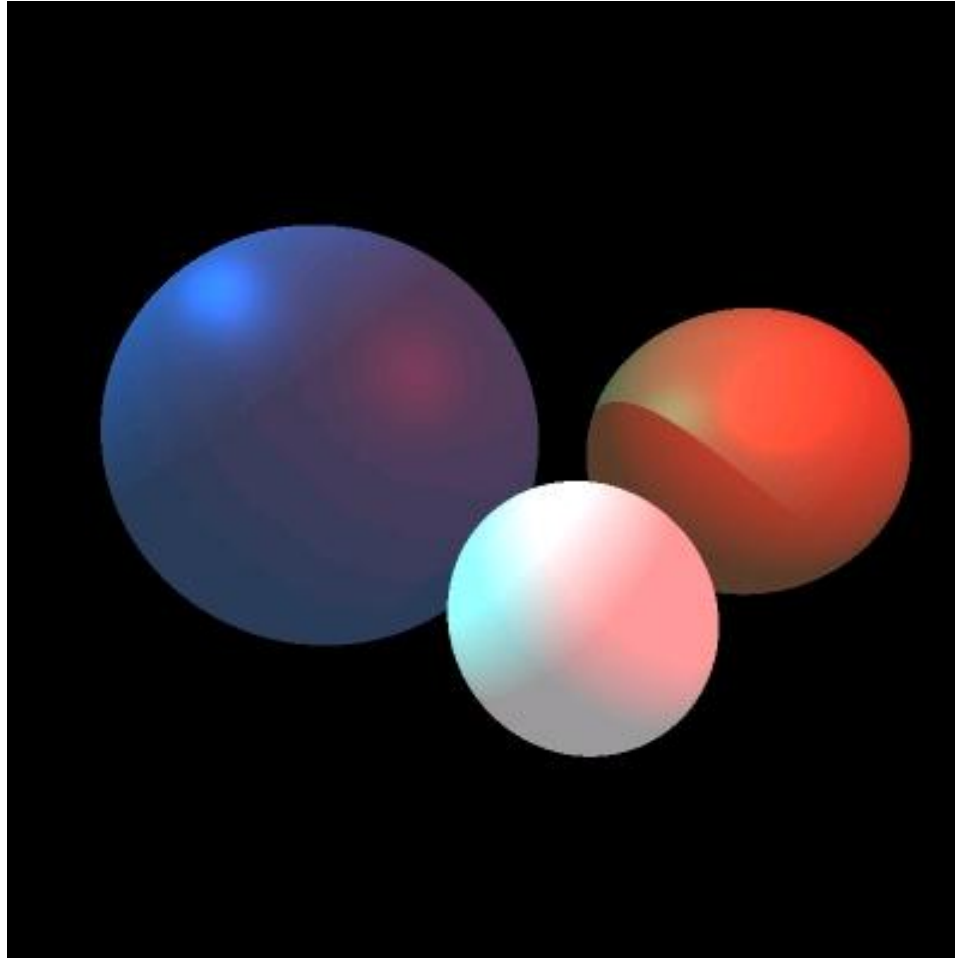
Correcting for Non-Visible Lights



$$I_r = k_a I_a + \sum S_j I_{i,j} \left(k_d (n \cdot l_j) + k_s (h_j \cdot n)^m \right)$$

- *Where S_j is the result of intersecting the ray L with the scene objects*
- *Note consider your intersection points along the ray L carefully*
 - *Hint – they might be beyond the light!*

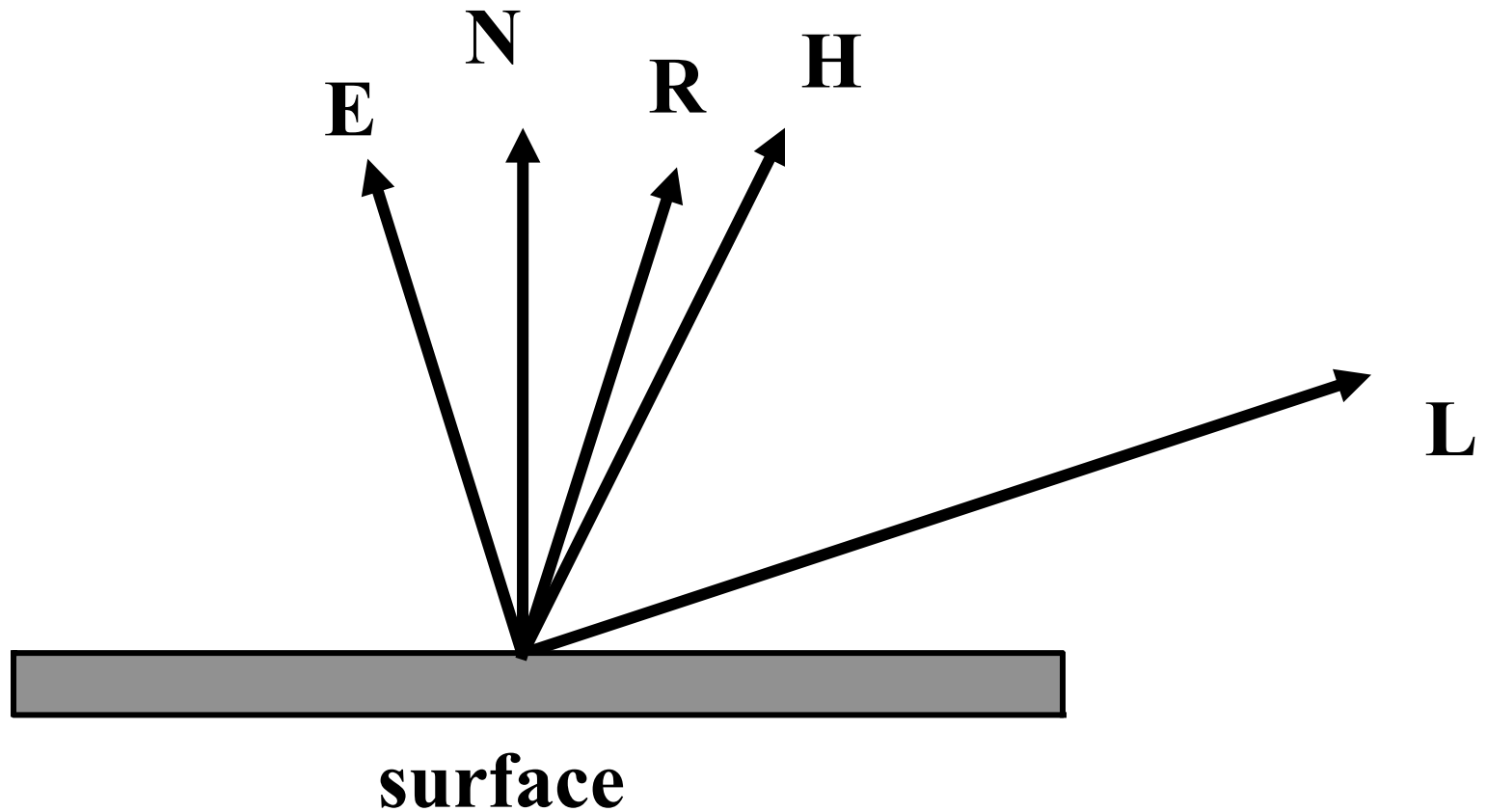
Result of Shadow Feeler



Recursive Ray-Tracing

- *We can simulate specular-specular transmission elegantly by recursing and casting secondary rays from the intersection points*
- *We must obviously chose a termination depth to cope with multiple reflections*

Introducing Reflection



■ Where
$$R = -E + 2(N \cdot E)N$$

Computing Reflectance

$$I_r = I_{local} + k_r I_{r'}$$

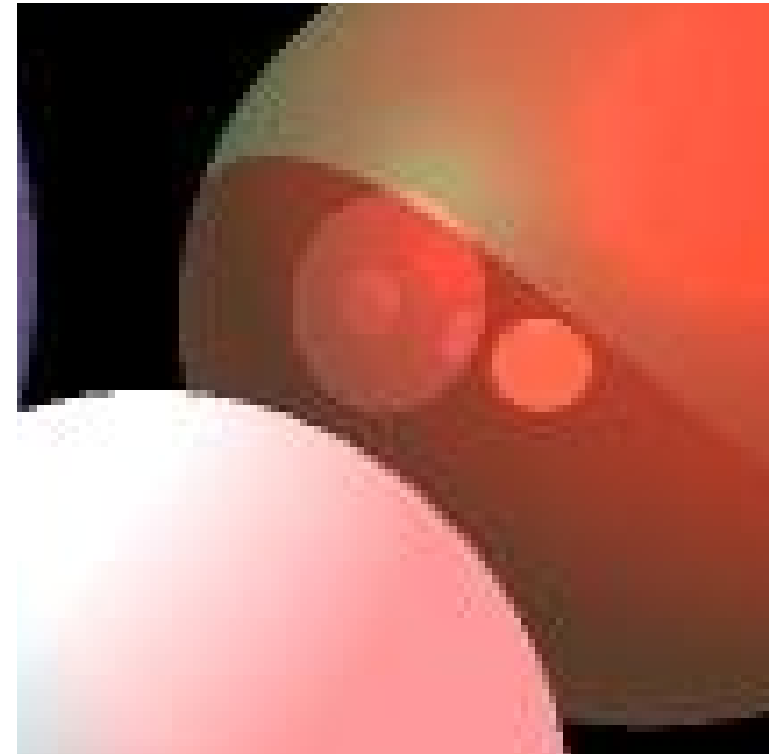
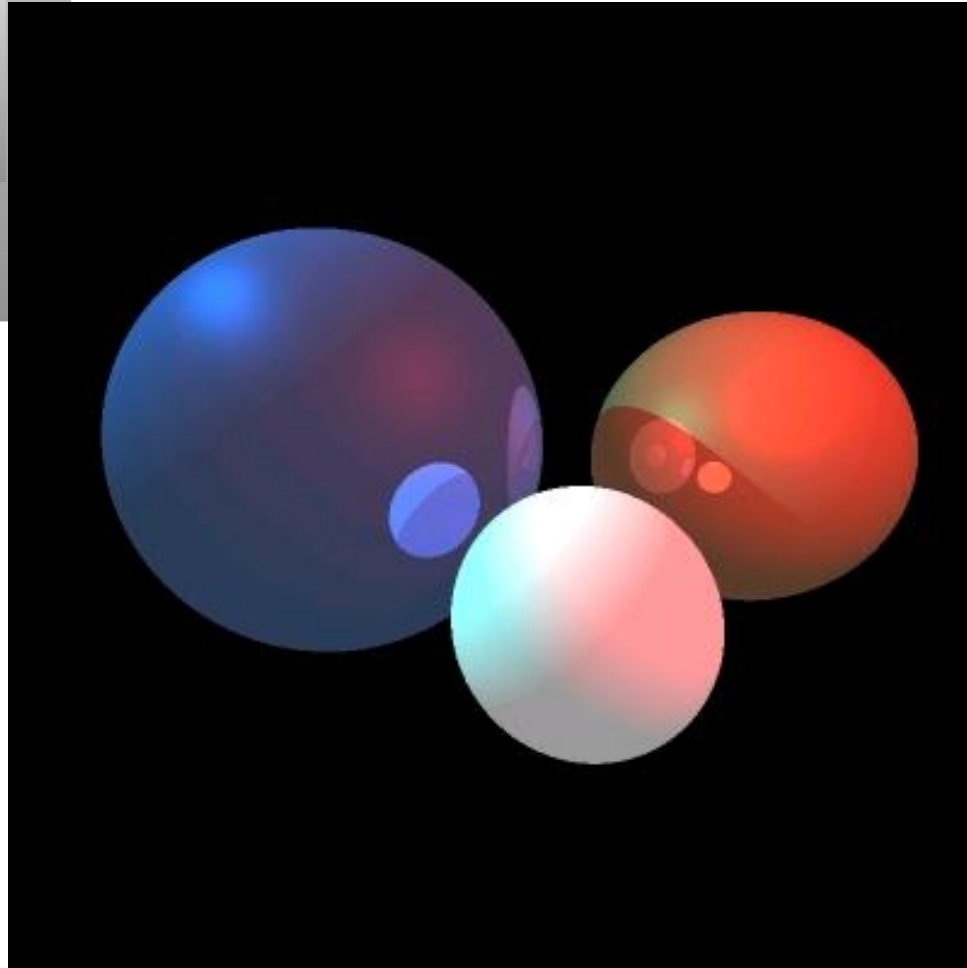
- *Where I_{local} is computed as before*
- *Ray r' is formed from intersection point and the direction R and is cast into the scene as before*

Pseudo Code

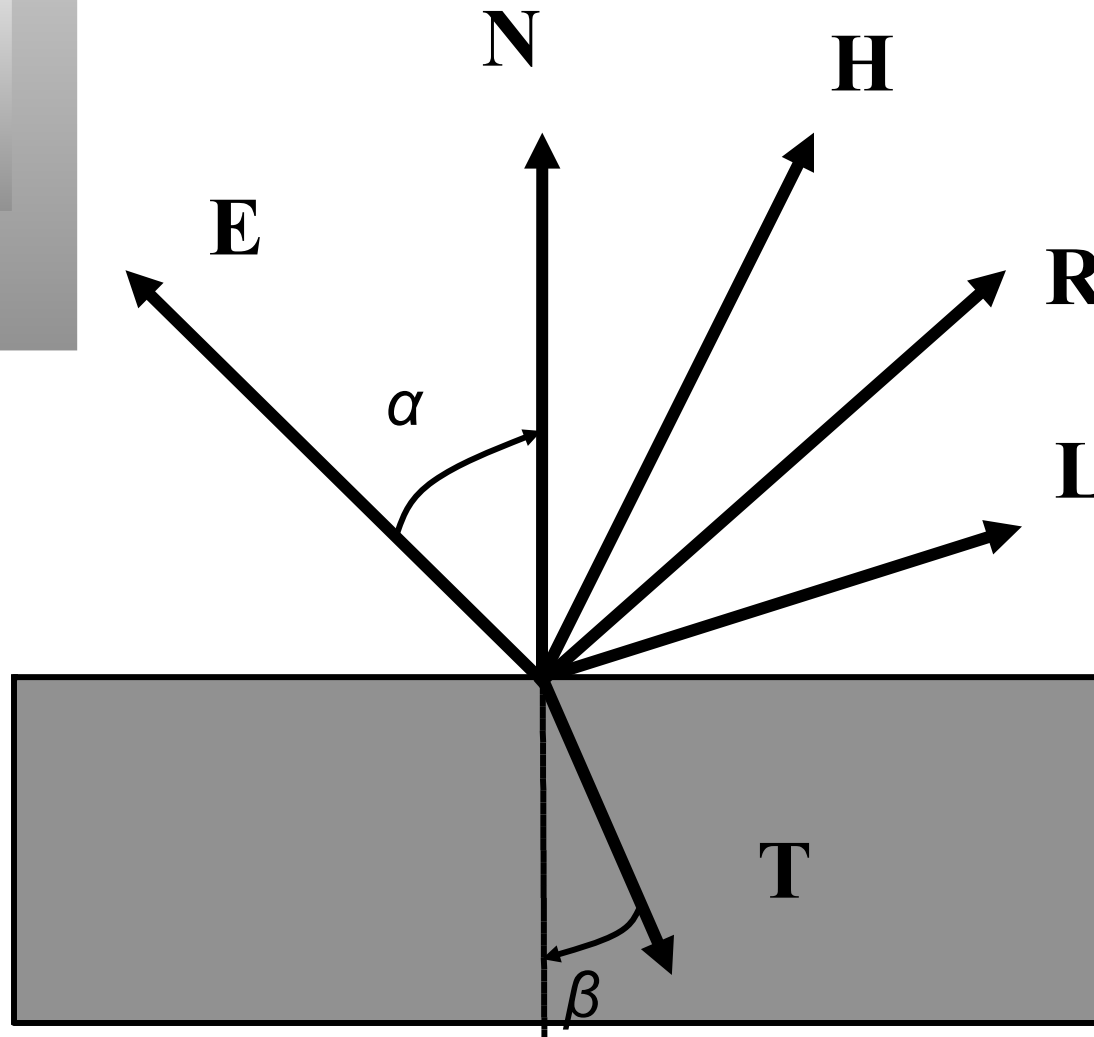
```
Color RayTrace(Point p, Vector D, int depth) {  
    Point pd    /* Intersection point */  
    Boolean intersection  
    if (depth > MAX) return Black  
    intersect(p,direction, &pd, &intersection)  
    if (!intersection) return Background  
     $I_{local} = k_a I_a + I_p.v.(k_d(n.l) + k_s.(h.n)^m)$   
    return  $I_{local} + k_r * RayTrace(pd, R, depth+1)$   
}
```

Normally $k_r = k_s$

Result of Recursion



Perfect Specular Transmission



Snell's Law

$$\frac{\sin \alpha}{\sin \beta} = \frac{\eta_2}{\eta_1}$$

*η is index of
refraction*

Using Snell's Law

$$\frac{\sin \alpha}{\sin \beta} = \frac{\eta_2}{\eta_1} = \eta_{21}$$

- *Using this law it is possible to show that:*

$$T = -\eta_{12}E + N \left(\eta_{12} \cdot \cos \alpha - \sqrt{1 + \eta_{12}^2 \cdot (\cos^2 \alpha - 1)} \right)$$

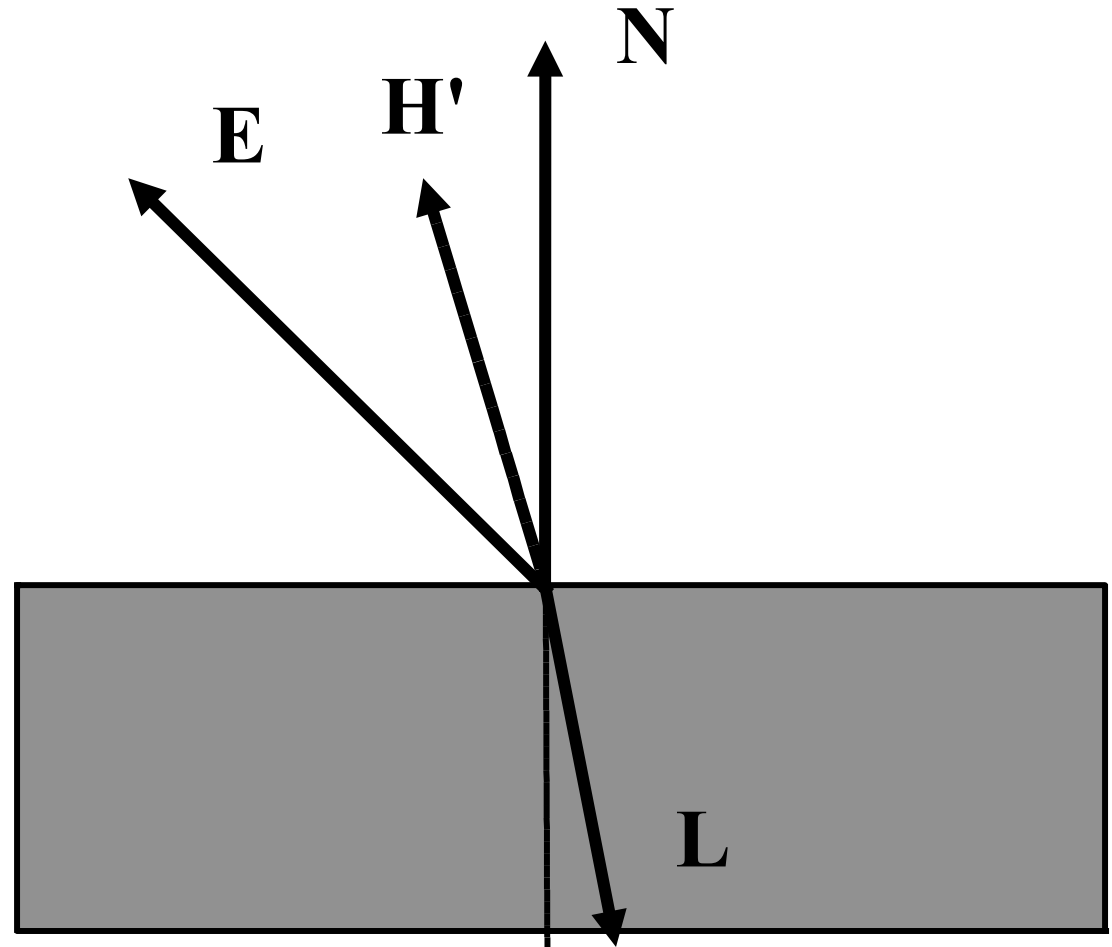
- *Note that if the root is negative then total internal reflection has occurred and you just reflect the vector as normal*

New Pseudo Code

```
Color RayTrace(Point p, Vector D, int depth) {  
    Point pd    /* Intersection point */  
    Boolean intersection  
    if (depth > MAX) return Black  
    intersect(p,direction, &pd, &intersection)  
    if (!intersection) return Background  
    
$$I_{local} = k_a I_a + I_p.v.(k_d(n.l) + k_s.(h.n)^m)$$
  
    return  $I_{local} + k_r * RayTrace(pd, R, depth+1) +$   
         $k_t * RayTrace(pd, T, depth+1)$   
}
```


Direct Specular Transmission

- *A transparent surface can be illuminated from behind and this should be calculated in I_{local}*



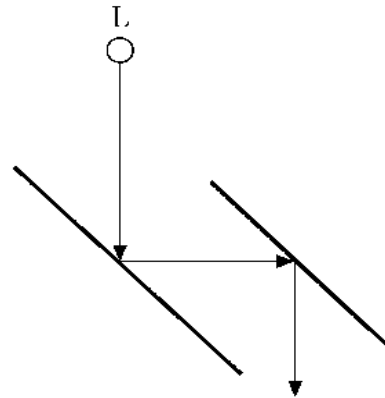
Calculating H'

$$H' = \frac{E - \frac{\eta_2}{\eta_1} L}{\frac{\eta_2}{\eta_1} - 1}$$

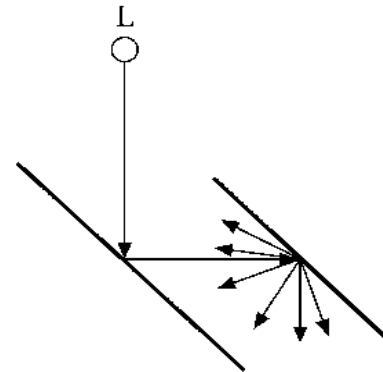
- *Use H' instead of H in specular term*

The diagram shows a 3D coordinate system with axes t_1 and t_2 . A line labeled 'viewplane' is shown with a series of tick marks. A line labeled 'COP' is also shown. Three points are marked: L1, L2, and B. Vectors r_1 and r_2 are shown originating from B. Arcs A, B, and C are shown. A vector s_1 is shown originating from B. A vector t_1 is shown originating from B. A vector t_2 is shown originating from B. A vector r_1 is shown originating from B. A vector r_2 is shown originating from B. A vector s_1 is shown originating from B. A vector t_1 is shown originating from B. A vector t_2 is shown originating from B.

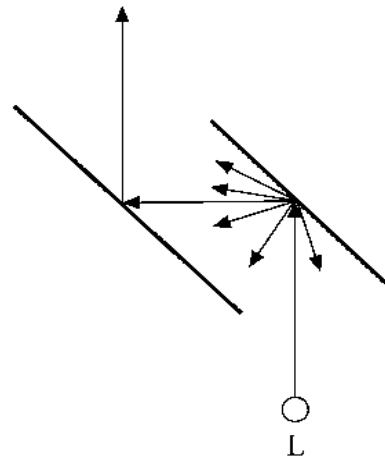
Discussion – What Can't We Simulate?



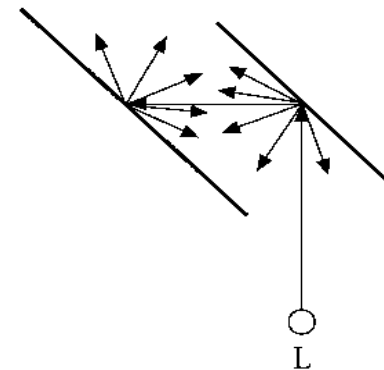
(a) specular to specular



(b) specular to diffuse



(c) diffuse to specular



(d) diffuse to diffuse

Summary

- *Recursive ray tracing is a good simulation of specular reflections*
- *We've seen how the ray-casting can be extended to include shadows, reflections and transparent surfaces*
- *However this is a very slow process and still misses some types of effect!*